

Reinforcement Learning Game Assignments

Choose ONE of the 12 game assignments in this document. Develop a simple playable game, train one reinforcement learning agent, and explain its performance. Each option carries 20 marks using the same rubric.

What you need to do

- Build or adapt a small 2D game with a visible objective, score, reset button or key, and basic human controls.
- Define the agent's observations, actions, and reward. Use one suitable RL algorithm; suggested choices appear with each game.
- Train the agent and provide a mode that loads and runs the trained policy. A saved Q-table is acceptable.
- Test the trained agent for 10 episodes and a random-action agent for 10 episodes using the same starting scenarios. Report average reward and the game metric listed in your option.
- Briefly explain one successful behavior, one limitation, and one possible improvement.

Keep the scope small

One level and simple shapes are sufficient. Use compact numerical observations rather than camera images. Existing game environments, physics engines, and RL libraries are allowed if acknowledged. Explain your own changes and the algorithm you used.

You do not need multiple RL algorithms, advanced graphics, online deployment, extensive parameter searches, or multiple training seeds. Extra features do not earn additional marks beyond the rubric.

What to submit

- Source code (python notebook), a saved policy or Q-table, and a short README with setup and run instructions.
- A report of 2–3 pages containing your information (Assignment title, Roll, Name), the game objective, observations/actions/reward, algorithm and main settings, a training reward plot, the 10-episode comparison, and a short discussion.
- A demonstration of about 3 minutes, live or recorded, showing human play and trained-agent play.

The following options describe the minimum game scope. All games are simulations; real hardware is not required.

How to submit

- Submit the above files to rakib@bau.edu.bd
- Use Google colab for developing the RL games.

Submission deadline

- 30 September, 2026

Assignment 1 Lunar Lander

Land a spacecraft safely on a platform using limited fuel.

Real world connection: Autonomous landing and vehicle control.

Build: Use one flat landing pad, gravity, and a few discrete thruster controls. End an episode on landing, crashing, or a time limit.

Observations: Position, velocity, tilt, and fuel.

Actions: Main thrust, left thrust, right thrust, or no thrust.

Suggested algorithm: DQN, or Q-learning with discretized observations.

Reward idea: Reward safe landing; penalize crashes and fuel use. Choose and explain simple numerical values.

Report: Number of safe landings out of 10. Also report average episode reward for the trained and random agents.

Assignment 2 Drone Delivery

Fly a drone around obstacles and reach a delivery zone.

Real world connection: Autonomous delivery.

Build: Use a small 2D map with two fixed obstacles and one target. End on reaching the target, collision, or a time limit.

Observations: Drone position, velocity, and target direction.

Actions: Move left, right, up, down, or hover.

Suggested algorithm: DQN.

Reward idea: Reward reaching the target; penalize collisions and each time step. Choose and explain simple numerical values.

Report: Successful deliveries out of 10. Also report average episode reward for the trained and random agents.

Assignment 3 Space Docking

Guide a spacecraft into a docking area at low speed.

Real world connection: Precision robotic docking.

Build: Use a stationary docking port and simple motion with inertia. No orbital mechanics are required. End on safe docking, collision, or a time limit.

Observations: Relative position and velocity.

Actions: Thrust left, right, up, down, or coast.

Suggested algorithm: DQN.

Reward idea: Reward low-speed docking; penalize impacts and unnecessary thrust. Choose and explain simple numerical values.

Report: Safe dockings out of 10. Also report average episode reward for the trained and random agents.

Assignment 4 Warehouse Robot

Collect a package and take it to a delivery point.

Real world connection: Warehouse automation.

Build: Use a small grid with fixed shelves, one package, and one destination. Pickup and delivery may happen automatically on entering the correct cell.

Observations: Robot cell and whether it carries the package.

Actions: Move up, down, left, or right.

Suggested algorithm: Tabular Q-learning.

Reward idea: Reward delivery; penalize blocked moves and each time step. Choose and explain simple numerical values.

Report: Completed deliveries out of 10. Also report average episode reward for the trained and random agents.

Assignment 5 Traffic Commander

Control traffic lights to move cars through an intersection.

Real world connection: Traffic signal control.

Build: Use one intersection with two traffic phases. The game must enforce safe transitions. Each episode lasts a fixed number of simulation steps.

Observations: Queue lengths and current signal phase.

Actions: Keep the current phase or request the other phase.

Suggested algorithm: Q-learning with grouped queue lengths, or DQN.

Reward idea: Reward cars crossing; penalize waiting queues. Choose and explain simple numerical values.

Report: Average vehicle waiting time across 10 episodes. Also report average episode reward for the trained and random agents.

Assignment 6 EcoRacer

Drive a car around a simple track without leaving the road.

Real world connection: Autonomous driving.

Build: Use one top-down track at a fixed forward speed. End on completing a lap, leaving the track, or a time limit.

Observations: Distance from track center, heading error, and nearby track boundaries.

Actions: Steer left, steer right, or continue straight.

Suggested algorithm: DQN.

Reward idea: Reward forward progress; penalize leaving the track. Choose and explain simple numerical values.

Report: Completed laps out of 10. Also report average episode reward for the trained and random agents.

Assignment 7 BalanceBot

Keep a robot upright while moving toward a finish line.

Real world connection: Robot balance and stabilization.

Build: Use a simple cart-and-pole style robot on flat ground. End on reaching the finish while upright, falling, or a time limit.

Observations: Position, velocity, tilt, and angular velocity.

Actions: Apply force left or right.

Suggested algorithm: DQN.

Reward idea: Reward staying upright and reaching the finish; penalize falling. Choose and explain simple numerical values.

Report: Successful crossings out of 10. Also report average episode reward for the trained and random agents.

Assignment 8 Rescue Maze

Reach a stranded person and return to the exit.

Real world connection: Search and rescue navigation.

Build: Use a small visible grid maze with fixed walls, one person, and one exit. Rescue is automatic when the agent returns with the person.

Observations: Agent cell and whether the person has been collected.

Actions: Move up, down, left, or right.

Suggested algorithm: Tabular Q-learning.

Reward idea: Reward rescue; penalize blocked moves and each time step. Choose and explain simple numerical values.

Report: Completed rescues out of 10. Also report average episode reward for the trained and random agents.

Assignment 9 Ocean Explorer

Collect an underwater sample and return to base.

Real world connection: Underwater robot navigation.

Build: Use a small grid with fixed rocks and a gentle current that occasionally shifts the robot one cell. End on return with the sample or a time limit.

Observations: Robot cell and sample status.

Actions: Move up, down, left, or right.

Suggested algorithm: Tabular Q-learning.

Reward idea: Reward returning the sample; penalize blocked moves and each time step. Choose and explain simple numerical values.

Report: Samples returned successfully out of 10. Also report average episode reward for the trained and random agents.

Assignment 10 Power Grid Defender

Keep a small city supplied with electricity using a battery.

Real world connection: Energy storage management.

Build: Create a turn-based game with 24 turns, a finite battery, and a simple repeating supply and demand pattern. Display city lights and battery charge.

Observations: Turn number, battery charge, supply, and demand.

Actions: Charge, discharge, or hold, within battery limits.

Suggested algorithm: Tabular Q-learning with discrete energy levels.

Reward idea: Penalize unmet demand and unnecessary battery use. Choose and explain simple numerical values.

Report: Average number of turns with unmet demand per episode. Also report average episode reward for the trained and random agents.

Assignment 11 Storekeeper Quest

Keep a game shop stocked and earn profit over a short season.

Real world connection: Inventory management.

Build: Create a turn-based shop with one product, limited storage, next-turn delivery, and random daily demand. Each episode lasts 30 days.

Observations: Stock level and day number.

Actions: Order zero, a small quantity, or a large quantity.

Suggested algorithm: Tabular Q-learning.

Reward idea: Use daily sales revenue minus purchase, holding, and shortage costs. Choose and explain simple numerical values.

Report: Average total profit across 10 episodes. Also report average episode reward for the trained and random agents.

Assignment 12 Recycling Rush

Move a sorting paddle to send falling items into the correct bins.

Real world connection: Automated waste sorting.

Build: Use two item categories and two bins. Present one item at a time with a limited number of steps before it falls. Each episode contains 20 items.

Observations: Item category, paddle position, and time until the item falls.

Actions: Move left, move right, or stay.

Suggested algorithm: Tabular Q-learning.

Reward idea: Reward correct sorting; penalize incorrect or missed items. Choose and explain simple numerical values.

Report: Percentage of items sorted correctly across 10 episodes. Also report average episode reward for the trained and random agents.

Marking rubric

Criterion	Marks	How marks are awarded
Game functionality	5	Game runs with an objective and reset (2); human controls and visible score or status (1); correct game rules and episode ending (2).
RL formulation and training	6	Suitable observations and actions explained (2); numerical reward explained (1); working RL training and a saved policy or Q-table (3).
Testing and results	4	Runs 10 trained-agent and 10 random-agent episodes on matching starting scenarios (2); reports average reward and the specified game metric for both (1); includes a readable training reward plot (1).
Short report and code clarity	3	Clear explanation of the algorithm and main training settings (1); discusses a success, a limitation, and an improvement (1); readable code, run instructions, and acknowledgement of reused code or libraries (1).
Demonstration	2	Shows human play (1) and the saved RL agent playing (1).
Total	20	